Programming for AI	Lab 02 Python Data Structures, Functions and Data Handling

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2026

AIMS AND OBJECTIVES

1. To understand the concept and importance of Python's built-in data structures.
2. To learn how to create, access, modify, and manipulate lists, dictionaries, tuples, and sets using built-in methods and operations.
3. To understand the characteristics and appropriate use cases of lists, dictionaries, tuples, and sets for different data-handling tasks.
4. To perform common data operations such as insertion, deletion, searching, sorting, indexing, slicing, and iteration using Python data structures.
5. To learn how to use built-in methods and functions associated with lists, dictionaries, tuples, and sets for efficient data processing.
6. To understand the performance characteristics of Python data structures and compare their efficiency for common operations such as access, search, insertion, and deletion.

INTRODUCTION

In this lab we will be covering the following topics.

Python provides several powerful built-in data structures that are essential for storing, organizing, and processing data efficiently. The four primary data structures—lists, dictionaries, tuples, and sets—are widely used in Python programming because each is designed for different data-handling requirements. Lists are ordered and mutable collections, tuples are ordered and immutable, dictionaries store data in key-value pairs, and sets store unique elements. Understanding their built-in methods, operations, use cases, and performance characteristics enables programmers to select the most appropriate data structure for a particular problem and write efficient, readable, and maintainable Python programs.

1.Lists

A **list** is an ordered and mutable collection in Python that can store multiple values of different data types. Lists allow duplicate elements and support indexing, slicing, modification, and dynamic resizing. They are commonly used when data needs to be stored in a specific order and modified during program execution.

Function/Method	Purpose
len()	Returns the number of elements
append()	Adds an element at the end
insert()	Adds an element at a specific position
extend()	Adds multiple elements
remove()	Removes a specified element
pop()	Removes and returns an element
clear()	Removes all elements
index()	Returns the index of an element
count()	Counts occurrences of an element
sort()	Sorts the list
reverse()	Reverses the list
copy()	Creates a copy of the list

Code Examples

Creating and accessing a list:

```
students = ["Ali", "Sara", "Ahmed", "Ayesha"]

print(students)
print(students[0])
print(students[-1])
```

Adding and removing elements:

```
students.append("Hamza")
students.insert(1, "Zain")

students.remove("Ali")
students.pop()

print(students)
```

List slicing:

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

```
print(numbers[:3])
print(numbers[::2])
```

Sorting a list:

```
marks = [78, 92, 65, 88, 71]

marks.sort()

print(marks)
```

List comprehension:

```
numbers = [1, 2, 3, 4, 5]

squares = [x ** 2 for x in numbers]

print(squares)
```

2. DICTIONARIES

A dictionary is a mutable Python data structure that stores data in key-value pairs. Each key is unique and is used to access its corresponding value. Dictionaries are particularly useful when data needs to be identified using meaningful keys instead of numerical indexes.

2.2 Common Dictionary Functions and Methods

Function/Method	Purpose
len()	Returns the number of key-value pairs
keys()	Returns all keys
values()	Returns all values
items()	Returns key-value pairs
get()	Retrieves a value safely
update()	Adds or updates elements
pop()	Removes a specified key
popitem()	Removes the last key-value pair
clear()	Removes all elements

copy()	Creates a copy

2.4 Code Examples

Creating and accessing a dictionary:

```
student = {
    "name": "Ali",
    "age": 22,
    "department": "Computer Science",
    "marks": 85
}
print(student["name"])
print(student["marks"])
```

Adding and updating values:

```
student["semester"] = 4
student["marks"] = 90

print(student)
```

Using get():

```
print(student.get("name"))
print(student.get("email", "Email not found"))
```

Using keys(), values(), and items():

```
print(student.keys())
print(student.values())
print(student.items())
```

Dictionary iteration:

```
for key, value in student.items():
    print(key, ":", value)
```

Dictionary comprehension:

```
numbers = [1, 2, 3, 4, 5]

squares = {x: x ** 2 for x in numbers}
```

```
print(squares)
```

3. TUPLES

A tuple is an ordered and immutable collection in Python. Like lists, tuples can contain multiple values and allow duplicate elements, but their elements cannot be changed after the tuple is created. Tuples are useful when data should remain constant throughout program execution.

3.2 Common Tuple Functions and Methods

Function/Method	Purpose
len()	Returns the number of elements
count()	Counts occurrences of an element
index()	Returns the index of an element
min()	Returns the smallest value
max()	Returns the largest value
sum()	Calculates the sum of numeric elements

3.4 Code Examples

Creating and accessing a tuple:

```
student = ("Ali", 22, "Computer Science", 85)

print(student)
print(student[0])
print(student[2])
```

Tuple slicing:

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Tuple methods:

```
numbers = (10, 20, 20, 30, 40)

print(numbers.count(20))
print(numbers.index(30))
```

Tuple unpacking:

```
student = ("Ali", 22, "Computer Science")

name, age, department = student

print(name)
print(age)
print(department)
```

Immutability:

```
numbers = (10, 20, 30)

# numbers[0] = 100
# This will generate a TypeError
```

4. SETS

A set is an unordered and mutable collection that stores unique elements**.** Sets automatically eliminate duplicate values and provide efficient membership testing. Python sets also support mathematical operations such as union, intersection, difference, and symmetric difference.

4.2 Common Set Functions and Methods

Function/Method	Purpose
len()	Returns the number of elements
add()	Adds an element
update()	Adds multiple elements
remove()	Removes an element
discard()	Removes an element without raising an error if absent
pop()	Removes an arbitrary element
clear()	Removes all elements
union()	Combines two sets
intersection()	Finds common elements
difference()	Finds elements present in one set but not another

symmetric_difference()	Finds elements present in either set but not both

4.4 Code Examples

Creating a set:

```
numbers = {10, 20, 30, 40, 50}

print(numbers)
```

Removing duplicates:

```
numbers = [10, 20, 20, 30, 30, 40]

unique_numbers = set(numbers)

print(unique_numbers)
```

Adding and removing elements:

```
fruits = {"apple", "banana", "mango"}

fruits.add("orange")
fruits.remove("banana")

print(fruits)
```

Union and intersection:

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

print(set_a.union(set_b))
print(set_a.intersection(set_b))
```

Difference:

```
print(set_a.difference(set_b))
print(set_b.difference(set_a))
```

Membership testing:

```python
students = {"Ali", "Sara", "Ahmed"}

if "Sara" in students:
    print("Student found")
else:
    print("Student not found")
```

5. COMPARISON OF PRIMARY DATA STRUCTURES

Feature	List	Dictionary	Tuple	Set
Ordered	Yes	Yes*	Yes	No
Mutable	Yes	Yes	No	Yes
Duplicates	Allowed	Keys: No	Allowed	Not allowed
Indexing	Yes	By key	Yes	No
Main Purpose	Ordered collection	Key-value data	Fixed collection	Unique values
Common Access	Index	Key	Index	Membership
Example	[1, 2, 3]	{"a": 1}	(1, 2, 3)	{1, 2, 3}

*Modern Python dictionaries preserve insertion order, but their primary conceptual model is key-value mapping, not sequence indexing.

5.1 Performance Overview

For common operations, the approximate average-case complexities are:

Operation	List	Dictionary	Tuple	Set
Access	O(1)	O(1)	O(1)	—
Search	O(n)	O(1)	O(n)	O(1)
Insert	O(1)**	O(1)	—	O(1)
Delete	O(n)	O(1)	—	O(1)
Membership	O(n)	O(1)	O(n)	O(1)

** list.append() is O(1) amortized; inserting into the middle of a list is generally O(n).

This comparison helps demonstrate why choosing the correct data structure is important when designing efficient Python programs.

LAB Tasks:

1. A university maintains student names and their marks in different subjects. You need to develop a program that can store the data, calculate each student's average, identify the highest-performing student, and generate a list of students who achieved an average above a specified threshold. **Requirement:** The solution should handle multiple students and multiple subjects efficiently.

2. A small e-commerce system maintains information about its products, including product ID, name, category, price, and available quantity. The system should support product lookup, price updates, stock updates, and identification of products that are out of stock. **Requirement:** Design the data representation so that product information can be accessed efficiently using a unique identifier.

3.A payment system receives a large collection of transaction IDs. Due to network problems, some transactions may appear more than once.Develop a solution that identifies duplicate transaction IDs and produces a collection containing only unique transactions. **Requirement:** Consider the performance implications when processing thousands of transaction IDs.

4.A university has two groups of students enrolled in two different courses.

- Course A contains one group of student IDs.
- Course B contains another group of student IDs.

The university wants to determine:

- Students enrolled in both courses
- Students enrolled only in Course A
- Students enrolled only in Course B
- All unique students across both courses

Requirement: Choose a data structure that naturally supports these operations.

5.An organization stores employee information such as employee ID, name, department, salary, and job title.Create a system capable of handling operations such as searching for an employee, updating salary information, adding new employees, and removing employees who leave the organization.

Requirement: The system should prioritize efficient employee lookup rather than sequential searching.

6.A server generates thousands of log entries containing information such as:

INFO, ERROR, WARNING, INFO , ERROR, INFO

Develop a program that analyzes the logs and determines:

- How many times each log type occurred
- Which log types appeared
- The most frequently occurring log type

Requirement: The solution should work efficiently as the number of log entries increases.

7.An online shopping application allows customers to add products to a shopping cart, remove products, modify quantities, and calculate the total price.A product may appear only once in the cart, but its quantity can change.Design a suitable data representation for the shopping cart and implement the required operations. **Requirement:** Think carefully about whether a simple list is sufficient for this problem.

8.You receive the following dataset containing user email addresses:

```
emails = [
   "ali@gmail.com", "sara@yahoo.com", "ali@gmail.com",  "ahmed@gmail.com",  "sara@yahoo.com",
   "zain@hotmail.com"
]
```

The dataset contains duplicate records. Develop a data-cleaning solution that produces a collection of unique email addresses while preserving the original data when necessary. **Requirement:** Consider the trade-off between uniqueness, ordering, and performance**.**

9.A software application contains configuration information that should not be accidentally modified during program execution.

Example information may include:

- Application name
- Version
- Supported environments
- Database configuration

Design a suitable data structure for storing information that should remain unchanged.**Requirement:** Explain why your selected structure is preferable to a mutable alternative and demonstrate what happens when an attempt is made to modify it

10.A company provides the following information:

```
employees = [
   ("E101", "Ali", "IT", 85000),  ("E102", "Sara", "HR", 75000),  ("E103", "Ahmed", "IT", 95000),
   ("E104", "Zain", "Finance", 90000)
]
```

Develop a data-processing solution that can answer questions such as:

- Which employees belong to the IT department?
- What is the average salary?
- Who has the highest salary?
- Which departments exist?
- How many employees are in each department?
- Can an employee be efficiently retrieved using their employee ID? **Requirement:** You are expected to use more than one Python data structure and justify why each structure was selected.